

HYDRA ETL / LIVRE BLANC

# HYDRA DSL

## VERSION 1.0

Guide de référence des mots-clés, workflows,
jobs, actions, paramètres et secrets

Compatible avec Hydra 0.16.1

hydraetl.com | github.com/bejaouibechir/Hydra

# Sommaire

01 Workflows

02 Jobs

03 Actions

04 Paramètres et secrets

## À propos de cette édition

Cette publication reprend la référence des mots-clés Hydra DSL version 1.0 fournie dans le document source, avec des exemples de configuration et une explication du rôle de chaque instruction.

Les exemples sont présentés dans des cadres à coloration syntaxique, afin de faciliter leur lecture et leur réutilisation.

# 1. Workflows

Fichiers : workflows/<nom>.yaml et workflows/thread.yaml.

## 1.1 Le workflow

### version : version du format DSL

```
version: "1.0"   # toujours "1.0" aujourd'hui
```

Rôle. permet au moteur de valider le fichier avec la bonne grammaire.

### workflow : racine d'un workflow

```
workflow:        # tout le contenu du workflow est sous cette clé
  name: daily_sales
```

Rôle. déclare un enchaînement d'étapes (jobs et actions).

### name : nom du workflow (obligatoire)

```
workflow:
  name: daily_sales   # identifiant affiché dans les logs et le fichier undo
```

Rôle. identifie le workflow ; sert aussi de nom pour hdrctl workflow undo.

### description : texte libre

```
description: "Charge les ventes du jour"
```

Rôle. documentation uniquement, sans effet à l'exécution.

### trigger : déclencheur (défaut manual)

```
trigger: manual     # manual | schedule | webhook
```

Rôle. indique quand le workflow démarre.

### cron : planification (obligatoire avec trigger: schedule)

```
trigger: schedule
cron: "0 6 * * *"   # tous les jours à 06:00
```

Rôle. expression cron à 5 champs donnant l'heure de démarrage.

### rollback : annulation en cas d'échec (défaut false)

```
rollback: true      # si une étape échoue, les étapes déjà faites sont annulées
```

Rôle. déclenche les undo des étapes terminées, dans l'ordre inverse.

### steps : liste des étapes (obligatoire)

```
steps:
  - name: load
    type: job
    job: ../jobs/orders
```

Rôle. contient les étapes exécutées par le workflow.

## 1.2 L'étape (step)

### name : nom unique de l'étape (obligatoire)

```
- name: load_orders   # sert de référence dans depends_on
```

Rôle. identifie l'étape et permet aux autres de dépendre d'elle.

### type : nature de l'étape (obligatoire)

```
type: job           # job | action
```

Rôle. indique si l'étape exécute un job ou une action.

### job : chemin du job (avec type: job)

```
type: job
job: ../jobs/orders   # dossier contenant sources/destinations/pipeline
```

Rôle. désigne le dossier du job à lancer.

### action : nom de l'action (avec type: action)

```
type: action
action: log           # une des 11 actions (voir section 3)
```

Rôle. choisit l'action à exécuter.

### params : paramètres de l'action

```
action: log
params:
  message: "Début du chargement"
```

Rôle. fournit les entrées de l'action ; accepte {{ param: }} et ${SECRET:}.

### depends_on : étapes à attendre (défaut [])

```
- name: publish
  depends_on: [load, clean]   # démarre quand load ET clean sont finies
```

Rôle. construit le graphe d'exécution ; les étapes sans dépendance commune tournent en parallèle.

### on_failure : comportement en cas d'échec (défaut fail)

```
on_failure: continue   # fail | skip | continue
```

Rôle. fail arrête le workflow ; skip ignore les étapes dépendantes ; continue les exécute quand même.

### enabled : active ou désactive l'étape (défaut true)

```
enabled: false   # l'étape est sautée sans être supprimée
```

Rôle. désactiver temporairement une étape.

### when : condition d'exécution

```
when: "{{ param:publish_enabled }} == True"   # exécutée seulement si vrai
```

Rôle. exécute l'étape selon une expression sûre (paramètres, résultat d'une étape).

### retry : politique de nouvelle tentative

```
retry:
  max: 3             # tentatives après le premier échec
  delay: 10          # secondes d'attente
  backoff: exponential   # fixed | exponential (délai doublé)
```

Rôle. absorbe les pannes passagères (réseau, base indisponible).

### undo : comment annuler l'action

```
type: action
action: bash
params: { command: "touch /tmp/flag" }
undo:
  action: bash
  params: { command: "rm /tmp/flag" }   # exécuté si rollback: true
```

Rôle. obligatoire pour webhook, email, bash, powershell, ssh, python (ou undo: none).

## 1.3 Le thread

### thread : racine de thread.yaml

```
thread:            # liste d'étapes (stages)
  - mode: sequential
    workflows:
      - name: load_referentials
```

Rôle. fixe l'ordre d'exécution de tous les workflows du projet (hdrctl run .).

### mode : exécution d'un stage (défaut sequential)

```
mode: parallel     # sequential | parallel
```

Rôle. les workflows du stage tournent l'un après l'autre ou en même temps.

### delay_after : pause après le stage

```
delay_after: 5m    # attente avant le stage suivant
```

Rôle. laisse le temps à un système externe de se stabiliser.

### workflows : workflows du stage (au moins un)

```
workflows:
  - name: load_sales
  - name: load_stock
```

Rôle. liste ce que le stage exécute (workflows/<name>.yaml).

### active : active ou saute un workflow (défaut true)

```
- name: load_stock
  active: false
```

Rôle. mise de côté sans modifier le fichier du workflow.

### delay_before : attente avant le démarrage

```
- name: build_reports
  delay_before: 30s
```

Rôle. décaler le démarrage d'un workflow.

### timeout : durée maximale

```
- name: load_sales
  timeout: 20m     # dépassé = échec
```

Rôle. empêche un workflow de bloquer le thread.

### fallback : workflow de remplacement

```
- name: load_sales
  fallback:
    use: [load_sales_backup]       # workflows essayés dans l'ordre
    when: "not source_available('crm')"
    on_failure: true               # remplace aussi après un échec
    on_probe_error: keep           # keep | replace si la condition est invérifiable
```

Rôle. bascule vers un plan B quand la condition est vraie.

### use / when / on_failure / on_probe_error : clés de fallback

**Voir ci-dessus. Rôle : use liste les remplaçants, when décide de la bascule avant le démarrage, on_failure l'étend aux échecs, on_probe_error gère le cas où la condition ne peut pas être évaluée.**

### loops : rejouer une partie du thread

```
loops:
  - from: load_stock    # dernier workflow de la partie
    to: load_sales      # premier workflow de la partie
    times: 3            # nombre total d'exécutions (1 à 1000)
    delay: 10m          # pause avant chaque rejeu
```

Rôle. répète une séquence (polling, traitements par vagues).

### Fonctions des conditions de when (fallback)

| Fonction | Rôle |
|---|---|
| source_available('id') | la source répond |

| Fonction | Rôle |
|---|---|
| destination_available('id') | la destination répond |
| file_exists('path') | le fichier existe |
| file_size('path') | taille, comparée à 10KB, 5MB, 1GB |
| file_age('path') | ancienneté, comparée à 30s, 5m, 24h, 7d |
| url_ok('url') | réponse HTTP 2xx/3xx |
| secret_defined('NAME') | le secret existe |
| db.load('id') | requêtes actives sur la base |
| previous.status | success · failed · skipped · inactive · none |
| previous.rows | lignes écrites par le workflow précédent |

```
fallback:
  use: [load_sales_light]
  when: "db.load('dwh') > 50 or file_size('in/orders.csv') > 1GB"
```

# 2. Jobs

Dossier : jobs/<nom>/ avec sources.yaml, destinations.yaml, pipeline.yaml, transformations.yaml (optionnel).

## 2.1 Sources (sources.yaml)

### sources : racine

```
version: "1.0"
sources:
  orders:           # "orders" est l'id de la source
    type: csv
```

Rôle. déclare au moins une source, référencée ensuite par son id.

### type : connecteur (obligatoire)

```
type: postgresql   # csv, json, parquet, mysql/mariadb, postgresql/postgres, sqlserver/mssql,
mongodb
```

Rôle. choisit le connecteur de lecture ; insensible à la casse.

### connection : paramètres de connexion

```
connection:
  host: db.local
  port: 5432
  user: etl
  password: ${SECRET:db.password}   # jamais en clair
  database: sales
```

Rôle. indique comment joindre la source ({} pour un fichier local). Clés SQL : host, port, user, password, database, schema ; SQL Server : instance, tds_version, login_timeout, timeout.

### extract : quoi lire (obligatoire)

```
extract:
  table: orders.csv        # fichier, table, ou...
  batch_size: 5000         # lignes par lot (défaut 10000, minimum 10)
```

Rôle. définit la lecture ; doit contenir table, query ou collection.

### table / query / collection

```
extract: { table: public.orders }                      # table ou fichier
extract: { query: "SELECT * FROM orders WHERE total>0" } # SQL uniquement
extract: { collection: orders }                         # MongoDB
```

Rôle. désigne l'objet lu.

### filter et limit : MongoDB

```
extract:
  collection: orders
  filter: { status: "paid" }   # filtre de requête
  limit: 1000                  # nombre maximal de documents
```

Rôle. restreint la lecture côté base.

### Options CSV (dans connection)

```
connection:
  base_path: ./data   # dossier de résolution de table (défaut : dossier du job)
  delimiter: ";"      # défaut ,
  quotechar: '"'
  encoding: utf-8
  lineterminator: "\n"
  strict_schema: false
```

Rôle. adapter la lecture à la forme du fichier.

### schema : structure attendue (MongoDB)

```yaml
schema:
  mode: infer_strict          # auto | manual | infer_strict | hybrid
  drift_policy: warn          # fail | warn | ignore | adapt
  validation_policy: strict   # strict | best_effort | permissive
  sample_size: 100            # documents échantillonnés
  fields:
    - name: total
      type: float             # string, integer, float, decimal, boolean, date, datetime,
      array, object, json
      path: amount.total
      required: true
      default: 0
      array_handling: flatten # keep | flatten | explode | json
      description: "Montant"
```

Rôle. fixe la forme des documents et la réaction face à un champ nouveau (dérive).

## 2.2 Destinations (destinations.yaml)

### destinations : racine

```yaml
destinations:
  dwh_orders:        # id de la destination
    type: postgresql
```

Rôle. déclare où écrire.

### table / collection : cible

```yaml
load: { table: dwh.orders }   # collection est un alias pour MongoDB
```

Rôle. table ou fichier de destination (obligatoire).

### mode : mode d'écriture (défaut append)

```yaml
load:
  mode: upsert    # append | replace | upsert
  key: [order_id] # obligatoire avec upsert
```

Rôle. append ajoute, replace vide puis écrit, upsert met à jour ou insère selon la clé.

```yaml
key : colonnes clés
```

**Voir ci-dessus. Rôle : identifie une ligne pour l'upsert.**

**batch_size : lignes par écriture (10 à 100000, défaut 10000)**

```
load: { batch_size: 20000 }
```

Rôle. règle le compromis mémoire / vitesse.

**atomic : tout ou rien**

```
load: { atomic: true }   # annule l'écriture entière en cas d'erreur
```

Rôle. évite une destination à moitié chargée.

## 2.3 Pipeline (pipeline.yaml)

**pipeline, from, to**

```
version: "1.0"
pipeline:
  from: orders        # id déclaré dans sources.yaml
  to: dwh_orders      # id déclaré dans destinations.yaml
```

Rôle. relie une source à une destination ; les transformations s'appliquent entre les deux.

## 2.4 Transformations (transformations.yaml)

**transformations : liste ordonnée d'opérations**

```
version: "1.0"
engine: auto          # auto | pandas | duckdb (voir plus bas)
transformations:
  - select: { columns: [id, country, amount] }
  - filter: { expr: "amount > 0" }
```

Rôle. décrit ce qui arrive aux données ; les opérations s'exécutent dans l'ordre.

### Colonnes

#### select : garde des colonnes.

```
- select:
    columns: [id, country, amount]   # les autres sont supprimées
rename : renomme.
- rename:
    mapping: { cust_id: customer_id }   # ancien: nouveau
cast : change le type (int, float, str, bool, datetime).
- cast:
    mapping: { amount: float, created: datetime }
```

### Lignes

#### filter : garde les lignes vraies.

```
- filter: { expr: "country == 'BE' and amount >= {{ param:min_amount }}" }
sort : trie (stable).
- sort: { by: [country, amount], ascending: [true, false] }
deduplicate : supprime les doublons.
- deduplicate: { columns: [customer_id], keep: last }   # first | last
fill_null : remplace les valeurs vides.
- fill_null: { columns: { country: "N/A", amount: 0 } }   # ou value: 0 pour toutes
trim : retire les espaces.
- trim: { columns: [name] }   # défaut : toutes les colonnes texte
clean : nettoie le texte.
- clean: { columns: [status], case: upper }   # none | lower | upper
```

### Calcul

#### calculate : crée une colonne.

```
- calculate: { column: total_ttc, expr: "amount * 1.2" }
script : Python restreint.
- script:
    inputs: [amount]
    outputs: { amount_x2: float }
    mode: vectorized          # vectorized | row
    code: |
      df["amount_x2"] = df["amount"] * params["factor"]   # params = paramètres du run
```

### Agrégation

#### aggregate : regroupe et calcule.

```
- aggregate:
    by: [country]
    agg: { total: "sum(amount)", n: "count(id)" }
```

### Combinaison (une seconde source déclarée dans sources.yaml)

### join : jointure.

```
- join:
    right: customers
    key: customer_id          # ou left_key / right_key
    how: left                 # inner | left | right | outer
merge : fusion/mise à jour.
- merge: { right: corrections, key: id, delete_unmatched: false }
union : empilement.
- union: { right: orders_archive, distinct: true }
```

### Restructuration

### pivot : lignes en colonnes.

```
- pivot: { index: [country], column: month, values: amount, aggfunc: sum }
unpivot : colonnes en lignes.
- unpivot: { id_vars: [id], value_vars: [jan, feb], var_name: month, value_name: amount }
transpose : inverse lignes et colonnes.
- transpose: { index_col: id, header_name: column }
engine : moteur de calcul (racine de transformations.yaml)
engine: duckdb   # auto (défaut) | pandas | duckdb
```

Rôle. auto choisit DuckDB au-delà de 2 000 000 de lignes pour les opérations globales (aggregate, sort, join right/outer). HYDRA_ENGINE l'emporte sur le fichier.

### Expressions (expr, when)

Opérateurs de comparaison, logiques (and, or, not) et arithmétiques ; évaluation sûre, sans code arbitraire.

# 3. Actions

### Une action est une étape type: action. Structure commune :

```
- name: mon_etape
  type: action
  action: <nom>
  params: { ... }
  undo: { ... }      # requis pour webhook, email, bash, powershell, ssh, python
```

### log : écrit un message

```
action: log
params: { message: "Chargement terminé" }   # défaut : "Step '<name>' executed"
```

Rôle. tracer la progression.

### delay : attend

```
action: delay
params: { seconds: 30 }   # ou duration
```

Rôle. laisser un système se stabiliser entre deux étapes.

### condition : évalue une condition et stocke le résultat

```
action: condition
params:
  name: is_big                  # paramètre qui reçoit le résultat
  mode: expr                    # expr | structured
  expr: "{{ param:rows }} > 1000"
  # structured : left / op / right
```

Rôle. alimenter les when des étapes suivantes.

### webhook : appel HTTP

```
action: webhook
params:
  url: https://hooks.example.com/notify
  method: POST                  # défaut POST
  headers: { Authorization: "Bearer ${SECRET:api.token}" }
  body: { status: done }        # mapping (JSON) ou texte
undo:
  action: webhook
  params: { url: https://hooks.example.com/cancel }
```

Rôle. notifier ou déclencher un système externe.

### email : envoie un e-mail

```
action: email
params:
  to: ops@example.com
  subject: "Rapport prêt"
  body: "Voir pièce jointe"
  from_addr: hydra@example.com  # défaut hydra@localhost
  smtp_host: smtp.example.com   # défaut localhost
  smtp_port: 587                # défaut 1025
  username: etl
  password: ${SECRET:smtp.password}
  use_tls: true                 # STARTTLS
undo: none
```

Rôle. alerter les personnes concernées.

### bash : commande shell

```
action: bash
params:
  command: "./scripts/archive.sh"
  working_dir: ./scripts        # défaut dossier courant
  timeout: 60                   # secondes, défaut 60
undo: { action: bash, params: { command: "./scripts/restore.sh" } }
```

Rôle. exécuter un traitement système (Linux/macOS).

### powershell : commande PowerShell

```
action: powershell
params: { command: "Copy-Item a.csv b.csv", timeout: 60 }
undo: { action: powershell, params: { command: "Remove-Item b.csv" } }
```

Rôle. équivalent de bash sous Windows.

### ssh : commande distante

```
action: ssh
params:
  host: srv1.example.com
  username: etl
  command: "systemctl restart app"
  port: 22                      # défaut 22
  key_path: ~/.ssh/id_ed25519   # utilisé en premier
  password: ${SECRET:ssh.password}  # si pas de clé
  timeout: 30
undo: none
```

Rôle. piloter un serveur distant.

### python : code Python

```
action: python
params:
  script: |                     # code en ligne, ou file_path: job.py
    print("bonjour")
  working_dir: .
  timeout: 60
undo: none
```

Rôle. logique spécifique que le DSL ne couvre pas.

### set_param : crée un paramètre pour la suite du run

```
action: set_param
params: { name: batch_label, value: "nightly", type: str }   # échoue si déjà défini
```

# 4. Paramètres et secrets

## 4.1 Déclaration

### parameters (fichier parameters.yaml, racine du projet)

```yaml
version: "1.0"
parameters:
  min_amount:
    type: float           # int | float | str/string | bool/boolean | json | any (défaut any)
    default: 50           # valeur si rien d'autre n'est fourni
    required: false       # true : le run s'arrête si aucune valeur
    description: "Seuil minimal"
```

Rôle. déclare les valeurs modifiables d'un projet, avec leur type.

### type / default / required / description

Voir ci-dessus. Booléens acceptés : true 1 yes on / false 0 no off.

## 4.2 Environnements

### environments/<env>.yaml

```yaml
# environments/prod.yaml
parameters:
  publish_enabled: true
  country: BE
```

Rôle. surcharge des paramètres par environnement (dev, test, prod).

### --env / HYDRA_ENV : choix de l'environnement

```
hdrctl run . --env prod        # ou HYDRA_ENV=prod
```

Rôle. sélectionne le fichier d'environnement.

### --param / -P : valeur pour un seul run

```
hdrctl run . --env prod -P min_amount=100 -P country=BE
```

Rôle. surcharge ponctuelle. Priorité : run > environnement > défaut.

## 4.3 Notations

### {{ param:NAME }} : valeur d'un paramètre

```
when: "{{ param:publish_enabled }} == True"
```

Rôle. injecte un paramètre dans jobs, params, when et scripts d'action. Erreur si inconnu ; garde son type s'il est seul.

### {{ env:NAME }} : variable d'environnement (tolérante)

```
base_path: "{{ env:DATA_DIR }}"   # pas d'erreur si absente
```

Rôle. lire une variable système sans bloquer.

### ${ENV:NAME} : variable d'environnement (stricte)

```
host: ${ENV:DB_HOST}   # run arrêté : "Missing environment variable: DB_HOST"
```

Rôle. exiger une variable.

### ${SECRET:NAME} : secret

```
password: ${SECRET:db.password}
```

Rôle. injecter une valeur sensible sans l'écrire dans le YAML. La valeur est masquée dans tout ce que Hydra affiche (voir 4.6).

## 4.4 Résolution des secrets

Pour ${SECRET:db.password}, dans l'ordre (la première source qui connaît le nom l'emporte) :

| # | Source | Exemple |
|---|---|---|
| 1 | variable d'environnement normalisée (. et - deviennent _, majuscules) | DB_PASSWORD |
| 2 | variable d'environnement au nom exact | db.password |
| 3 | fichier de secrets local, en clair ou chiffré par SOPS | .hydra/secrets |
| 4 | fournisseur déclaré dans .hydra/secrets.yaml (Hydra 0.17.0+) | provider: sops |

L'environnement (CI, shell, .env) l'emporte toujours : la production peut fournir toutes les valeurs sans aucun fichier.

### .hydra/secrets : fichier local

```
# nom=valeur, une par ligne, # = commentaire
db.password=change-me
api.token="abc 123"
export smtp.password=s3cret
```

Rôle. stockage local, créé par hdrctl init et ajouté à .gitignore. Si le fichier est chiffré par SOPS, Hydra le détecte et le déchiffre (voir 4.7).

### HYDRA_SECRETS_FILE / --secrets-file : autre emplacement

```
hdrctl run . --secrets-file /etc/hydra/secrets
```

Rôle. changer le fichier de secrets sans modifier le projet. En clair ou chiffré par SOPS.

### secret_defined('NAME') : tester un secret (dans when)

```
when: "secret_defined('api.token')"
```

Rôle. n'exécuter une étape que si le secret existe.

## 4.5 Réglages des secrets (Hydra 0.17.0+)

### .hydra/secrets.yaml : visibilité et fournisseur

```
# .hydra/secrets.yaml : réglages seulement, aucun secret (peut être commité)
visibility: masked                  # plain | masked | hidden (défaut masked)
provider: sops                      # none (défaut) | sops
sops:
  file: .hydra/secrets.sops.yaml    # fichier chiffré, relatif à la racine du projet
  format: yaml                      # dotenv | yaml | json (défaut : selon l'extension)
  age_key_file: ~/keys/hydra.txt    # optionnel (défaut : SOPS_AGE_KEY_FILE)
```

Rôle. choisit comment les secrets apparaissent dans les logs et d'où ils viennent. Clé, valeur ou fournisseur inconnus : le run s'arrête avec une erreur explicite. vault, azure-keyvault et aws-secrets-manager sont des noms réservés, pas encore disponibles.

## 4.6 Visibilité des secrets dans les logs (Hydra 0.17.0+)

| Mode | Ce qui est affiché | Usage |
|---|---|---|
| plain | la valeur, en clair ; un avertissement le rappelle | débogage uniquement |
| masked | [MASKED] à la place de la valeur | défaut |
| hidden | la ligne est omise et comptée : [1 line(s) hidden: they contain a secret] | audits stricts, logs partagés |

### --secrets-visibility / HYDRA_SECRETS_VISIBILITY : pour un run

```
hdrctl run . --secrets-visibility hidden
hdrctl workflow run load_sales --secrets-visibility plain   # ou HYDRA_SECRETS_VISIBILITY=plain
```

Rôle. surcharge le réglage du projet pour un run. Priorité : --secrets-visibility > HYDRA_SECRETS_VISIBILITY > visibility: > masked.

### Exemple

```
- name: notify
  type: action
  action: python
  params:
    script: "print('token=${SECRET:api.token}')"
# masked  ->  token=[MASKED]
# hidden  ->  [1 line(s) hidden: they contain a secret]
```

Rôle. le script reçoit toujours la vraie valeur ; seul ce qui est affiché ou conservé change.

### Ce qui est masqué

Les logs (tous les loggers, logs des étapes, API, sortie du CLI), les messages d'erreur et les traces, la sortie des actions bash, powershell, python et ssh, les lignes de log des actions webhook et email, le rapport d'annulation (undo) et les runs conservés par l'API.

Formes masquées : la valeur brute, sa forme encodée dans une URL (chaînes de connexion), sa forme échappée en JSON, et chaque ligne d'une valeur multiligne (clés PEM). Toutes les valeurs du fichier de secrets sont masquées, même si aucun ${SECRET:} ne les utilise.

Règles : une valeur de moins de 4 caractères n'est pas masquée et un avertissement la nomme ; si deux réglages divergent pour une même valeur, le plus strict l'emporte. Les valeurs envoyées aux connecteurs, scripts et webhooks ne sont jamais modifiées.

## 4.7 Secrets chiffrés avec SOPS et age (Hydra 0.17.0+)

### Créer la clé et chiffrer le fichier

```
age-keygen -o ~/.config/sops/age/keys.txt        # affiche la clé publique age1...
sops --encrypt --age age1... secrets.yaml > .hydra/secrets.sops.yaml
sops .hydra/secrets.sops.yaml                    # modifier : déchiffré dans l'éditeur seulement
```

Rôle. SOPS chiffre les valeurs, pas les clés : le fichier peut être commité et relu. Installer d'abord sops et age (Windows : winget install Mozilla.SOPS FiloSottile.age).

### secrets.yaml avant chiffrement

```
db:
  password: change-me     # lu par ${SECRET:db.password}
api:
  token: abc123           # lu par ${SECRET:api.token}
```

Rôle. les clés imbriquées sont jointes par des points. Ne jamais commiter ce fichier en clair : le supprimer une fois chiffré.

### Fonctionnement

Le fichier est déchiffré en mémoire au premier secret demandé ; le clair ne touche jamais le disque. sops choisit la clé (age, PGP, KMS cloud) ; avec age, il lit SOPS_AGE_KEY_FILE ou sops.age_key_file. HYDRA_SOPS_BIN donne le chemin de sops s'il n'est pas dans le PATH.

Hydra Studio refuse de modifier un fichier chiffré (le modifier avec sops). Erreurs explicites : "sops is not installed or not on PATH", "cannot decrypt ... no age key can open this file".

## 4.8 Transmission aux scripts

| Où | Comment le paramètre arrive |
|---|---|
| bash, powershell, ssh, python | variable d'environnement du nom du paramètre : $country, $env:country, os.environ["country"] |
| script de transformation | mapping en lecture seule params["min_amount"] |

Un secret n'arrive dans un script que par un ${SECRET:NAME} écrit dans ses params.

## 4.9 Fichiers .env

Le .env du dossier du job l'emporte sur celui de la racine du projet (priorité : job > racine).

Limites connues : un secret qu'un script lit directement dans l'environnement, sans ${SECRET:} ni fichier de secrets, n'est pas masqué ; une valeur transformée (base64, découpée) non plus.